35. Deque in Java — Quick Notes

Deque (**Double Ended Queue**) allows **insertion and deletion at both ends**.

What is Deque?

Deque = Double Ended Queue

Insert/Delete: ✓ Front ✓ Rear
Therefore, Deque can be used as both: Queue (FIFO) and Stack (LIFO)

Java Hierarchy

Collection
↑
Queue
↑
Deque (Interface)

Implementations

Implementation	Internal Structure
LinkedList	Doubly Linked List
ArrayDeque	Dynamic Circular Array

Java Terminology

Deque Concept	Java Name
Front	First
Rear	Last

Non-Throwing Methods

(Return special values on failure)

Operation	Method	Failure Return
Insert Front	`offerFirst(x)`	`false`
Insert Rear	`offerLast(x)`	`false`
Remove Front	`pollFirst()`	`null`
Remove Rear	`pollLast()`	`null`
Access Front	`peekFirst()`	`null`
Access Rear	`peekLast()`	`null`

Exception-Throwing Methods

(Throw exception on failure)

Operation	Method
Insert Front	`addFirst(x)`
Insert Rear	`addLast(x)`
Remove Front	`removeFirst()`
Remove Rear	`removeLast()`
Access Front	`getFirst()`
Access Rear	`getLast()`

Syntax

Using LinkedList

```
Deque<Integer> d = new LinkedList<>();
```

Using ArrayDeque (Preferred)

```
Deque<Integer> d = new ArrayDeque<>();
```

Example

```
Deque<Integer> d = new ArrayDeque<>();

d.offerFirst(10);
d.offerLast(20);

d.offerFirst(5);
d.offerLast(15);
```

State of Deque

After:
offerFirst(10);
offerLast(20);
offerFirst(5);
offerLast(15);

Deque becomes: First Last
5 10 20 15

---

peekFirst()

Returns first element.
Does NOT remove it.

```
System.out.println(d.peekFirst());
```

Output: 5

---

peekLast()
Returns last element.
Does NOT remove it.
System.out.println(d.peekLast());

Output: 15

---

pollFirst()
Removes and returns first element.
System.out.println(d.pollFirst());

Output: 5
Deque becomes: 10 20 15

---

pollLast()
Removes and returns last element.
System.out.println(d.pollLast());

Output: 15
Deque becomes: 10 20

Example Program
Deque<Integer> d = new ArrayDeque<>();

d.offerFirst(10);
d.offerLast(20);
d.offerFirst(5);
d.offerLast(15);

System.out.println(d.peekFirst());
System.out.println(d.peekLast());

d.pollFirst();
d.pollLast();

System.out.println(d.peekFirst());
System.out.println(d.peekLast());
Output:

5
15
10
20

Traversing Deque
Using Iterator (First → Last)
Iterator<Integer> it = d.iterator();

while(it.hasNext()){
System.out.print(it.next() + " ");
}
Output: 5 10 20 15

Using For-Each Loop

```
for(int x : d){
   System.out.print(x + " ");
}
```

Output: 5 10 20 15

Reverse Traversal

(Last → First)

```
Iterator<Integer> it = d.descendingIterator();

while(it.hasNext()){
   System.out.print(it.next() + " ");
}
```

Output:

15 20 10 5

Queue Functions Supported

Since: Deque extends Queue
Deque also supports:

Queue Method	Equivalent Deque Method
offer()	offerLast()
add()	addLast()
poll()	pollFirst()
remove()	removeFirst()
peek()	peekFirst()
element()	getFirst()

Recommendation

Prefer:
offerFirst()
offerLast()

pollFirst()
pollLast()

peekFirst()
peekLast()

instead of Queue methods because: They clearly indicate which end is used.

Deque as Stack

```
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);

System.out.println(stack.pop());
System.out.println(stack.peek());
```

Output:
20
10

Deque as Queue

```
Deque<Integer> queue = new ArrayDeque<>();

queue.offerLast(10);
queue.offerLast(20);

System.out.println(queue.pollFirst());
System.out.println(queue.peekFirst());
```

Output:
10
20

Time Complexity

LinkedList Implementation

Operations	Time Complexity
`offerFirst()`, `offerLast()`	**O(1)**
`pollFirst()`, `pollLast()`	**O(1)**
`peekFirst()`, `peekLast()`	**O(1)**
`addFirst()`, `addLast()`	**O(1)**
`removeFirst()`, `removeLast()`	**O(1)**
`getFirst()`, `getLast()`	**O(1)**

Interview Notes

```
// Deque supports insertion and deletion at both ends.
// Deque can act as both Queue and Stack.
// Prefer Deque-specific methods because they clearly
// indicate whether operations happen at first or last.
```

Quick Revision

Concept	Remember

Full Form	Double Ended Queue
Front in Java	First
Rear in Java	Last
Preferred Implementation	ArrayDeque
Non-Throwing Insert	offerFirst(), offerLast()
Non-Throwing Remove	pollFirst(), pollLast()
Non-Throwing Access	peekFirst(), peekLast()
Throwing Insert	addFirst(), addLast()
Throwing Remove	removeFirst(), removeLast()
Throwing Access	getFirst(), getLast()
Forward Traversal	iterator()
Reverse Traversal	descendingIterator()
Can act as	Queue + Stack
LinkedList Complexity	O(1) for both ends

Most Common Syntax

```
Deque<Integer> d = new ArrayDeque<>();

d.offerFirst(10);
d.offerLast(20);

d.pollFirst();
d.peekLast();
```

Deque is one of the **most versatile data structures in Java Collections** because it efficiently supports operations at **both ends**.